EXPLORE AI ACADEMY

Calculated columns with DAX

DAX data types, operators, and variables

DAX data types, operators, and variables

In DAX, we work with various **data types**, each tailored to specific **kinds of data**, such as numbers, text, and dates. It determines **how data can be manipulated** in calculations.

Operators in DAX, like arithmetic, comparison, string, and logical operators, allow us to **perform calculations** and **evaluate expressions**, shaping how we analyse and derive insights.

DAX **variables** enable us to **store intermediate results**, simplify complex expressions, and enhance formula readability and performance.

DAX data types

> Data types in DAX define the **kind of data** that can be stored in a column or used in expressions and calculations.

There are a few data types that we can use in DAX, but we will focus on the following:

Whole number	Decimal number	Boolean	Text
Integer values without fractions.	Real numbers with fractions.	True or false values.	Character strings.

Date	Currency	Blank
Dates and times.	Monetary values.	Represents an absence of data.

Using DAX with data

We will use the following `Country_stats` dataset to illustrate how we use DAX.

Country_ID	Region	Is_landlocked	GDP_per_capita_USD	Population_size	Area_km2	Survey_date
EGY	Northern Africa	FALSE	4,295.40	110,990,103	1,010,408.45	2019-07-16
ETH	Eastern Africa	TRUE	1,027.60	123,379,924	1,112,000.71	2022-07-28
GHA	Western Africa	FALSE	2,175.90	33,475,870	239,535.52	BLANK
KEN	Eastern Africa	FALSE	2,099.30	54,027,487	580,367.59	2022-02-01
MAR	Northern Africa	FALSE	3,527.90	37,457,971	446,300.34	2021-06-21
RWA	Eastern Africa	TRUE	966.3	13,776,698	26,338.50	2022-04-12
NGA	Western Africa	FALSE	2,184.40	218,541,212	923,769.19	2020-12-11
ZAF	Southern Africa	FALSE	6,776.50	59,893,885	1,221,037.55	2019-07-19

DAX data types

Data type	Description	Example
Whole number	Handles **integers** that **do not** have a decimal place. Used when decimals/fractions are not applicable or precision isn't required.	`Population_size`
Decimal number	Can handle numbers with **decimal places** and whole numbers. Needed for precise data.	`Area_km2`
Boolean	A value that is either **TRUE** or **FALSE**. Fundamental in creating filters or conditional statements in DAX.	`Is_landlocked`
Text	A **character** data string, which can be letters, numbers, or dates represented in a text format.	`Region`

DAX data types

Data type	Description	Example
Date	Represents a date, storing the day, month, and year.	`Survey_date`
Currency	Represents monetary values. Designed for financial calculations where accuracy and rounding rules are crucial.	`GDP_per_capita_USD`
Blank	Blank is a DAX data type that signifies an unknown or empty value. (It represents and replaces SQL nulls.)	Any empty values (look at `Survey_date`)

Operators in DAX

> Operators in DAX are **symbols** or **keywords** that are used to create expressions that compare values, perform arithmetic calculations, or work with strings.

Each operator type has a **specific function**, like arithmetic operators for mathematical calculations, comparison operators for filtering and sorting data, etc. Understanding operators is **crucial for building expressions** and **formulas**.

Think about the different data types in our `Country_stats` dataset. Do you think we can use all the operators with all the different types of data? Why or why not?

Arithmetic operators

> Arithmetic operators perform **mathematical calculations** and **produce numeric results**. They allow us to create calculated columns or measures that involve summation, subtraction, or other mathematical transformations of data.

+	-	*	/	^
Used to **add** two **numbers**.	Used to **subtract** one **number** from another.	Used for **multiplying** two **numbers**.	Used to **divide** one **number** by another.	Raises one number to the **power** of another.
Example: `1,027.60 + 2,175.90`	Example: `580,367.59 - 446,300.34`	Example: `580,367.59 * 3`	Example: `110,990,103 / 123,379,924`	Example: `239,535.52 ^ 2`

Comparison operators

> Comparison operators in DAX are used to **compare values**. When two values are compared by using these operators, the **result** is a logical value, either **TRUE** or **FALSE**.

=	==	>	>=	<	<=	<>
Checks if **A = B** Returns TRUE if value A in a column **equals** a specified value B.	Checks if **A = B** Returns TRUE if value A in a column and a specified B are **strictly equal**, i.e. have equal values or are both BLANK.*	Checks if **A > B** Returns TRUE where value **A** in a column is **greater than** a specified value **B**.	Checks if **A ≥ B** Returns TRUE where value **A** in a column is **greater than or equal to** a specified value **B**.	Checks if **A < B** Returns TRUE where value **A** in a column is **less than** a specified value **B**.	Checks if **A ≤ B** Returns TRUE where value **A** in a column is **less than or equal to** a specified value **B**.	Checks if **A ≠ B** Returns TRUE where value **A** in a column is **not equal** to a specified value **B**.

** = and == differ in handling BLANK: = returns TRUE when BLANK is compared to 0, an empty string, or FALSE; == returns TRUE only when BLANK is compared to BLANK.*

String operators

> The primary string operator in DAX is the **concatenation** operator (**&**), which **joins** two or more text strings into a **single string**.

*This operator is especially useful in **creating calculated columns** where data from different text columns need to be combined for enhanced insights.*

We could create a new column in our dataset that combines the country's `Country_ID` and `Region` into a single string, with a dash (-) between them.

DAX:
```
Country_region = [Country_ID] & " - " & [Region]
```

Country_region
EGY - Northern Africa
ETH - Eastern Africa
GHA - Western Africa
KEN - Eastern Africa
MAR - Northern Africa
RWA - Eastern Africa
NGA - Western Africa
ZAF - Southern Africa

Logical operators

Logical operator	Description	Example
&& (AND)	Returns TRUE if **both** conditions are TRUE; otherwise it returns FALSE.	**Check if a country is landlocked and has a population over 100 million:** `[Is_landlocked] && [Population_size] > 100000000`
|| (OR)	Returns TRUE if **at least one** of the expressions is TRUE; it's only FALSE if both expressions are FALSE.	**Check if a country is in either Eastern Africa or Northern Africa:** `[Region] = "Eastern Africa" || [Region] = "Northern Africa"`
IN	Checks if a value **exists** within a **specified list** or **table**, returning TRUE if a match is found or FALSE otherwise.	**Check if the country's region is in Northern Africa:** `[Region] IN {"Northern Africa"}`
NOT	**Reverses** the **logical value** of an expression, returning TRUE if the expression is FALSE, and vice versa. It is a unary operator, i.e. it operates on only one operand.	**Check if a country is not landlocked:** `NOT([Is_landlocked])`

Operator precedence

^	Exponentiation
-	Sign (as in –1)
* and /	Multiplication and division
+ and -	Addition and subtraction
&	Concatenation
=,==,<,>,<=,>=,<>,IN	Comparison
NOT	NOT (unary operator)

Order of operators

If we combine several operators in a single formula, the operations are **ordered** according to the table. If the operators have equal precedence value, they are ordered from **left** to **right**.

To change the order of evaluation, we can enclose the part of the formula that must be calculated first in parentheses.

Variables in DAX

> Variables are used to **store** the **result** of an expression as a **named** value. Defining a variable involves **assigning** an expression to a **name**, which can then be **used** throughout a DAX formula.

The need for variables in DAX

Variables are essential for **storing** and **reusing intermediate** results within expressions, eliminating the need for repeated calculations of the same expression within a formula.

Variables allow for a more **organised** and **efficient** way of handling complex calculations, thereby **simplifying** our DAX expressions and improving **performance**.

Defining variables

VAR and RETURN

To **define** a variable in DAX, we use the “**VAR**” keyword, followed by the **variable name** and the **expression** we want to assign to it. **The syntax is:** `VAR VariableName = Expression`

Once variables are defined, we use "**RETURN**" to **output** the formula's result, incorporating the defined variables.

Define a variable `Max_population` to store the maximum value from the `Population_size` column, and then return this value.

```
VAR Max_population = MAX([Population_size])
RETURN
Max_population
```

Advantages of using variables

Readability and clarity

Variables make DAX formulas **easier** to **read** and **understand**, especially when dealing with lengthy and complex expressions.

Code reusability

Once defined, a variable can be **reused** multiple times within a formula, which helps in keeping the code **concise** and **manageable**.

Optimise performance

By storing intermediate results, variables can **reduce** the **number** of calculations required, thus **optimising** the **performance** of the DAX expression.

Easier debugging

Variables **simplify** the debugging process, as they allow us to **isolate** and **inspect** parts of the formula, making it easier to **identify** and fix errors